

AULA 007 - Expressões, Atribuição e Controle

O que cada linha imprime?

main.py	+	Console
1	<code>print(-2 ** 2, 2 ** 3 ** 2)</code>	-4 512
2		

Main.java	+	Console
1	<code>import java.util.*;</code>	3345
2		
3	<code>public class Main {</code>	
4	<code> public static void main(String[] args) {</code>	
5	<code> System.out.println(1 + 2 + "3" + 4 + 5);</code>	
6	<code> }</code>	
7	<code>}</code>	

index.js	+	Console
1	<code>console.log("5" - 2, "5" + 2);</code>	3 52
2		

Main.c	+	Console
1	<code>#include <stdio.h></code>	0
2	<code>int main()</code>	
3	<code>{</code>	
4	<code> printf("%d\n", 3 > 2 > 1);</code>	
5		
6	<code>}</code>	

```
main.py x +
1 x = 0
2 print(x or "padrão", 3 > 2 > 1)
3
```

Console
padrão True

```
JS index.js x + Console I/O
1 console.log(0 == "", "" == "0", 0 == "0"); true false true
2
```

Quais são as regras de precedência?

A precedência define qual operador vai ser executado primeiro, normalmente operadores de multiplicação e divisão tem prioridade maior que soma e subtração

EX: $a + b * c$

Quais são as regras de associatividade?

A associatividade determina a ordem de avaliação quando dois operadores tem a mesma precedência, a maioria dos operadores associam da esquerda para a direita, mas alguns associam da direita para a esquerda.

Em que ordem os operandos são avaliados?

A linguagem precisa determinar em que ordem será os valores dos operandos de uma expressão, essa ordem pode ser importante quando existir chamadas de funções com efeitos colaterais.

EX:

$a + b * c$

Esquerda para direita: $(3 + 4) * 5 = 35$

Normal: $3 + (4 * 5) = 23$

Há restrições a efeitos colaterais?

Depende da linguagem, um efeito colateral ocorre quando a avaliação de uma expressão ou função altera algum valor como uma variável global, as linguagens podem definir uma ordem de avaliação para diminuir resultados imprevisíveis.

O usuário pode sobrecarregar operadores?

Depende da linguagem, algumas permitem que o programador atribua novos comportamentos a operadores existentes, por exemplo, o operador + pode ser definido para realizar uma operação entre objetos criados. outras linguagens não permitem esse tipo de sobrecarga

Que mistura de tipos é permitida?

A linguagem precisa definir se operandos de tipos diferentes podem aparecer na mesma expressão e se haverá conversões automáticas entre eles.

Precedência e associatividade na prática

A precedência define qual operação será feita primeiro e a associatividade define a ordem quando operadores tem a mesma precedência

A maioria dos operadores associa da esquerda para a direita.

EX: * / % + -

Já a exponenciação associa da direita para a esquerda.

EX: 2 ** 3 ** 2

Porque a exponenciação é feita antes do menos unário

O ideal é usar parênteses pois eles deixam a intenção do código mais clara.

Efeitos colaterais

Efeito colateral acontece quando uma função altera alguma variável ou estado externo. A ordem de avaliação pode mudar o resultado da expressão. Java avalia da esquerda para a direita, enquanto C pode não definir essa ordem.

Sobrecarga de operadores

É quando um operador possui mais de um significado. Exemplo: + pode somar números ou concatenar textos. Algumas linguagens, como Python, permitem que o programador defina novos comportamentos para operadores.

Conversão de tipos

Alargamento: conversão para um tipo maior.

Estreitamento: conversão para um tipo menor, podendo perder informação.

Coerção: conversão automática feita pela linguagem.

Cast: conversão explícita feita pelo programador.

EX: `double d = 3.99;`

`int i = (int) d; // 3`

Expressões booleanas

Em C: `3 > 2 > 1`

Resultado 0 porque primeiro `3 > 2` que é 1 e depois `1 > 1` que é falso

Em Python: `3 > 2 > 1`

Resultado True pois a comparação é encadeada

Curto-circuito

Ocorre quando não é necessário avaliar toda a expressão.

`s != null && s.length() > 0`

Se `s` for null, a segunda parte não será executada.

Outros operadores importantes:

`&& || ?. ??`

Atribuição

Atribuição utiliza `=` e também pode aparecer em formas compostas:

`+= -= *= /=`

Python permite:

`a, b = b, a`

e possui o operador morsa:

`:=`

Cuidado com:

`if (x = y)`

pois é uma atribuição, diferente de:

`if (x == y)`

Estruturas de controle

As três estruturas básicas são:

Sequência: comandos executados em ordem.

Seleção: escolha de um caminho com `if`, `else`, `switch` ou `match`.

Iteração: repetição com `for`, `while` e `do-while`.

Seleção

No `if/else`, o `else` pertence ao `if` mais próximo que ainda não possui `else`. Por isso, é recomendado utilizar `{ }`.

No `switch` de C, é necessário utilizar:

`Break`

para impedir que a execução continue para os próximos casos.

Rust utiliza:

`match`

que deve tratar todas as possibilidades.

Laços

Principais comandos:

for

while

do-while

Em C, alterar a variável de controle dentro do for pode mudar o funcionamento do laço.

Em Python:

for i in range(0, 10):

o range fornece os valores das próximas iterações.

break encerra o laço e continue passa para a próxima repetição.

for...in e for...of

Em JavaScript:

for...in

percorre índices e chaves.

for...of

percorre os valores.

Goto

goto realiza um desvio direto no programa, mas pode prejudicar a legibilidade. Por isso são preferidos comandos mais controlados:

break

continue

return

Comandos protegidos

Dijkstra propôs comandos baseados em condições chamadas guardas. Essa ideia aparece no select de Go:

```
select {  
    case ...  
    case ...  
}
```

Se mais de uma opção estiver disponível, uma delas é escolhida.

Pegadinhas importantes da aula

for...in

percorre índices, não valores.

d = d or 10

troca 0 por 10, pois 0 é considerado falso.

switch sem break pode continuar para os próximos casos.

if (condicao);

o ; encerra o if.

Em Go:

x := ...

pode criar uma nova variável dentro do bloco.

Em Java:

double taxa = 7 / 10 * 100;

resulta em 0.0, pois 7 / 10 é uma divisão inteira.